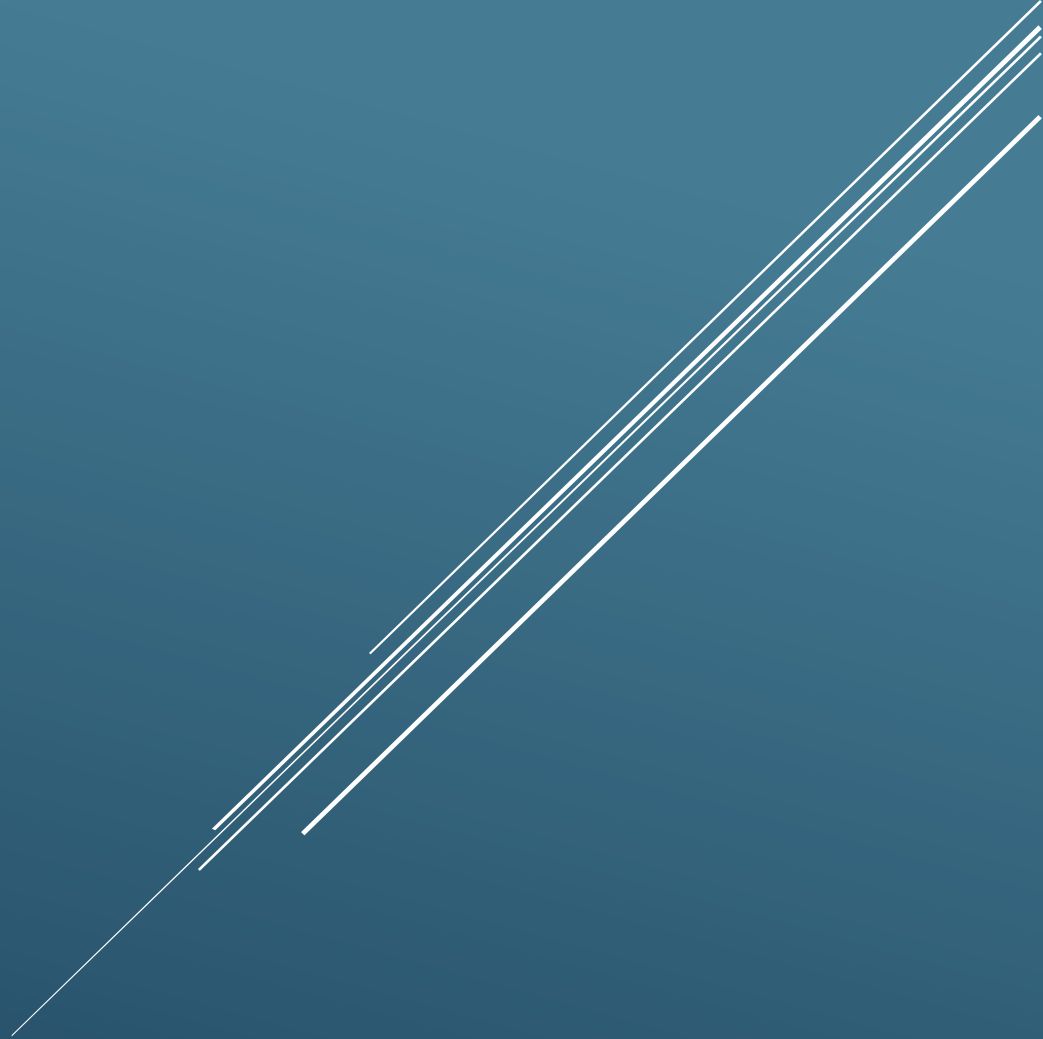




SA.6 — REPORTE DE CONFIGURACIÓN DE HERRAMIENTAS Y EMULADORES

Índice

Introducción.....	2
SA.6.A Configuración de herramientas.....	3
Flutter SDK y Dart SDK	3
Android Studio	3
Antigravity IDE	4
FFmpeg	5
Dependencias principales del proyecto.....	6
Aplicación de teléfono (Flutter).....	6
Aplicación Wear OS	7
Backend NestJS.....	8
Smart TV (Angular).....	9
Procedimiento de instalación	10
SA.6.B Configuración de emuladores	11
Emulador de teléfono	11
Emulador Wear OS	13
Emulación de Smart TV.....	15
Evidencias de funcionamiento.....	16
Problemas encontrados (Troubleshooting)	18
Problema 1. Variables de entorno en Render	18
Problema 2. Emulación de Bluetooth Low Energy	19
Conclusiones.....	19
Justificación del uso de Inteligencia Artificial	20

Introducción

Para el desarrollo del ecosistema inteligente de la barbería se utilizaron diferentes herramientas de software que permitieron implementar las tres aplicaciones que conforman el proyecto: la aplicación móvil para teléfonos Android, la aplicación para Wear OS y la PWA para Smart TV. Asimismo, se configuraron distintos emuladores para validar el funcionamiento del ecosistema sin necesidad de utilizar dispositivos físicos.

El objetivo de este documento es describir el entorno de desarrollo utilizado, las versiones de las herramientas empleadas, las dependencias principales de cada proyecto, la configuración de los emuladores y los problemas encontrados durante el desarrollo junto con sus respectivas soluciones.

SA.6.A Configuración de herramientas

Flutter SDK y Dart SDK

El desarrollo de las aplicaciones móviles (teléfono y Wear OS) se realizó utilizando Flutter como framework principal para el desarrollo multiplataforma.

Versión utilizada:

- Flutter SDK 3.44.0 (Stable)
- Framework revision: 559ffa3f75
- Engine revision: fcf463a224
- Dart SDK 3.12.0
- DevTools 2.57.0

Estas versiones garantizaron compatibilidad con Android 17 para la aplicación móvil y Android Wear OS 4 para la aplicación del reloj inteligente.

```
PS C:\Users\josue\barberiaBackup\dispositivos_inteligentes> flutter --version
Flutter 3.44.0 • channel stable • https://github.com/flutter/flutter.git
Framework • revision 559ffa3f75 (3 months ago) • 2026-05-15 14:13:13 -0700
Engine • hash fcf463a2242790d1fdcd9d044f533080f5022e18 (revision 4c525dac5e) (2 months ago) • 2026-05-15 19:00:04.000Z
Tools • Dart 3.12.0 • DevTools 2.57.0
```

Android Studio

Como entorno de desarrollo oficial para Android se utilizó Android Studio.

Configuración utilizada:

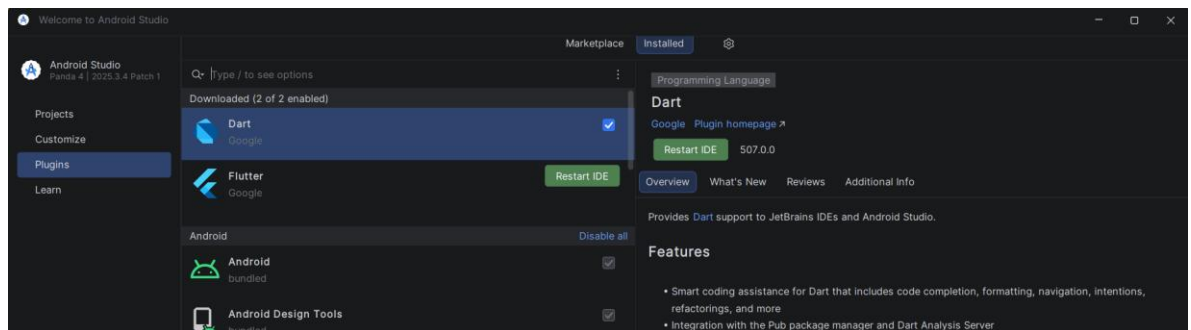
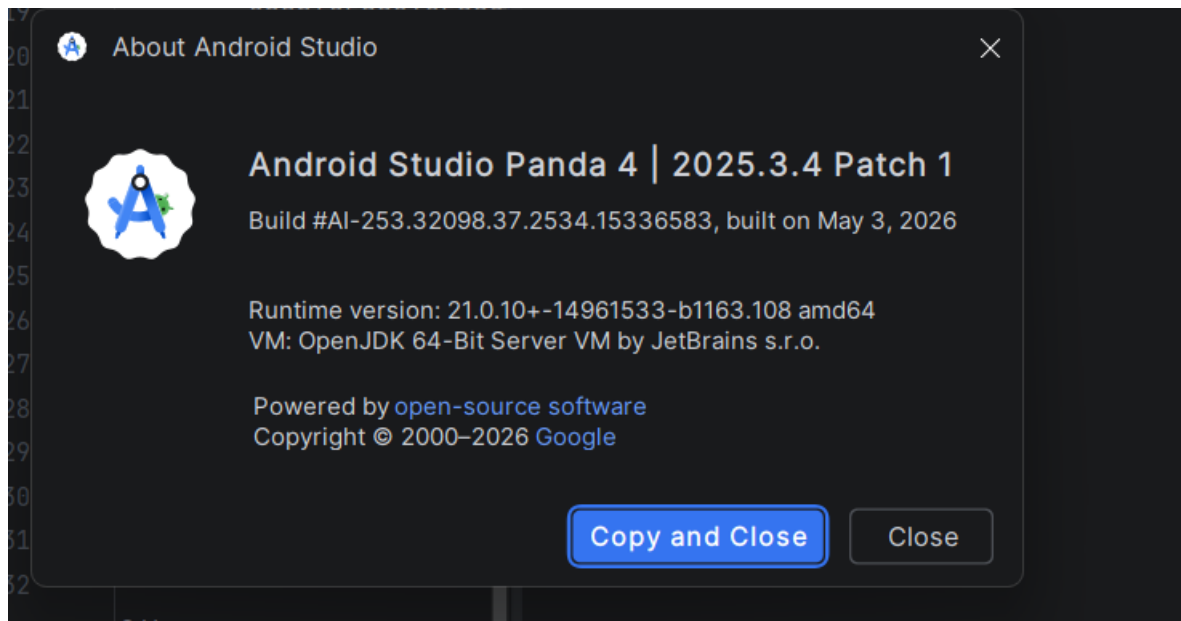
- Android Studio Panda 4 | 2025.3.4 Patch 1
- Build AI-253.32098.37.2534.15336583
- Runtime OpenJDK 21
- Sistema operativo Windows 11

Android Studio fue utilizado para:

- Desarrollo Flutter.
- Administración de emuladores.
- Compilación de APK.
- Ejecución de Wear OS.
- Administración del Android SDK.

Los plugins instalados fueron:

- Flutter
- Dart



Antigravity IDE

Antigravity IDE fue utilizado como editor principal durante el desarrollo del ecosistema.

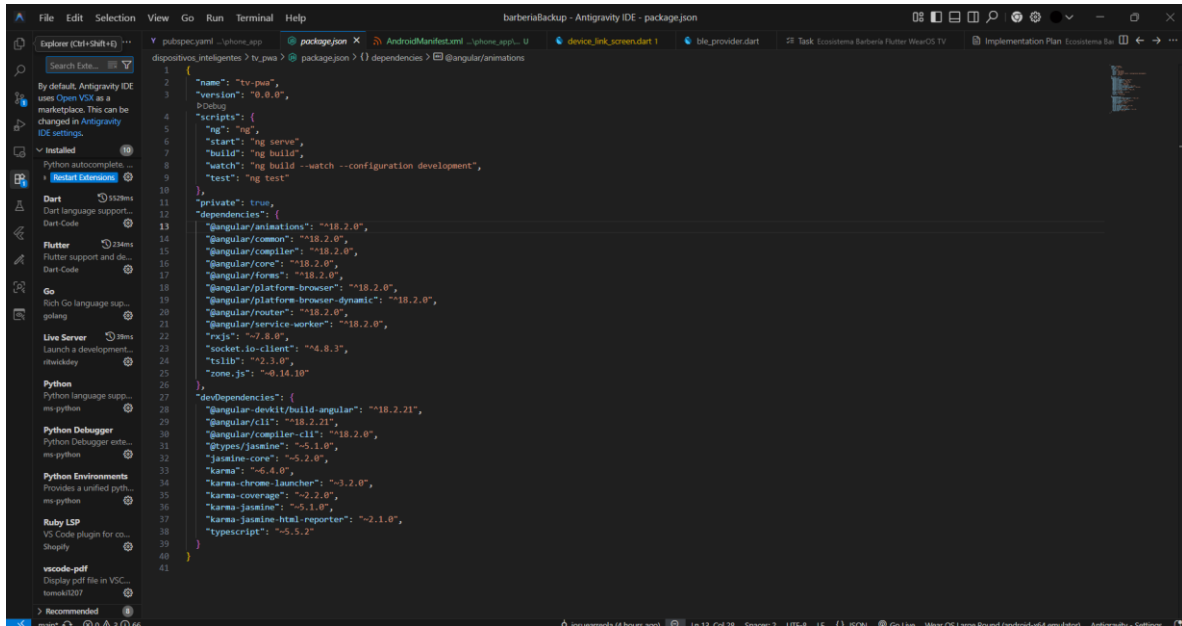
Se utilizó para:

- Desarrollo del Backend con NestJS.
- Desarrollo de la aplicación Flutter.
- Desarrollo de la PWA para Smart TV mediante Angular.

Las extensiones principales utilizadas fueron:

- Flutter
- Dart
- Angular Language Service

Estas extensiones facilitaron el autocompletado, análisis del código, depuración y soporte para los diferentes lenguajes utilizados durante el desarrollo.



FFmpeg

Se verificó la disponibilidad de FFmpeg dentro del entorno de desarrollo.

Resultado obtenido:

El comando `ffmpeg -version` no fue reconocido por el sistema operativo, por lo que esta herramienta no fue utilizada durante el desarrollo del proyecto.

Debido a que el proyecto no realiza procesamiento ni conversión de archivos multimedia, la ausencia de FFmpeg no afecta el funcionamiento del ecosistema desarrollado.

```
PS C:\Users\josue\barberiaBackup\dispositivos_inteligentes> ffmpeg -version
ffmpeg: El término 'ffmpeg' no se reconoce como nombre de un cmdlet, función, archivo de script o programa ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1
+ ffmpeg -version
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (ffmpeg:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

Dependencias principales del proyecto

Aplicación de teléfono (Flutter)

Las principales dependencias utilizadas fueron:

Dependencia	Función
provider	Administración del estado de la aplicación
http	Consumo de la API REST
flutter_secure_storage	Almacenamiento seguro del JWT
flutter_blue_plus	Comunicación BLE
socket_io_client	Comunicación en tiempo real mediante WebSockets
table_calendar	Calendario de citas
google_fonts	Tipografías personalizadas
permission_handler	Gestión de permisos del dispositivo

```
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  provider: ^6.1.2
  http: ^1.2.1
  flutter_secure_storage: ^9.2.2
  flutter_blue_plus: ^2.3.11
  intl: ^0.20.2
  google_fonts: ^6.2.1
  table_calendar: ^3.2.0
  permission_handler: ^13.0.0
  socket_io_client: ^3.1.6
  flutter_localizations:
    sdk: flutter

dev_dependencies:
  flutter_test:
    sdk: flutter

  # The "flutter_lints" package below contains a set of recommended lints to
  # encourage good coding practices. The lint set provided by the package is
  # activated in the `analysis_options.yaml` file located at the root of your
  # package. See that file for information about deactivating specific lint
  # rules and activating additional ones.
  flutter_lints: ^6.0.0
  flutter_launcher_icons: ^0.14.3
```

Aplicación Wear OS

Las dependencias principales fueron:

Dependencia	Función
flutter_ble_peripheral	Creación del periférico BLE
ble_peripheral	Publicación del servicio GATT
permission_handler	Permisos Bluetooth
socket_io_client	Simulación de comunicación en emuladores

```
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  flutter_ble_peripheral: ^2.1.1
  permission_handler: ^13.0.0
  ble_peripheral: ^2.4.0
  socket_io_client: ^3.1.6

dev_dependencies:
  flutter_test:
    sdk: flutter

  # The "flutter_lints" package below contains a set of recommended lints to
  # encourage good coding practices. The lint set provided by the package is
  # activated in the `analysis_options.yaml` file located at the root of your
  # package. See that file for information about deactivating specific lint
  # rules and activating additional ones.
  flutter_lints: ^6.0.0
  flutter_launcher_icons: ^0.13.1
```


Backend NestJS

Dependencias principales:

Dependencia	Función
@nestjs/common	Framework principal
@nestjs/jwt	Autenticación JWT
@nestjs/websockets	Comunicación Socket.IO
@nestjs/platform-socket.io	Gateway WebSocket
bcryptjs	Cifrado de contraseñas
helmet	Encabezados de seguridad
pg	Conexión con PostgreSQL Neon
typeorm	ORM

```
},  
"dependencies": {  
  "@nestjs/common": "^11.0.1",  
  "@nestjs/config": "^4.0.3",  
  "@nestjs/core": "^11.0.1",  
  "@nestjs/jwt": "^11.0.2",  
  "@nestjs/platform-express": "^11.0.1",  
  "@nestjs/platform-socket.io": "^11.1.28",  
  "@nestjs/throttler": "^6.4.0",  
  "@nestjs/typeorm": "^11.0.0",  
  "@nestjs/websockets": "^11.1.28",  
  "bcryptjs": "^2.4.3",  
  "class-transformer": "^0.5.1",  
  "class-validator": "^0.14.3",  
  "express-session": "^1.18.0",  
  "helmet": "^7.1.0",  
  "pg": "^8.18.0",  
  "reflect-metadata": "^0.2.2",  
  "resend": "^6.9.4",  
  "rxjs": "^7.8.1",  
  "socket.io": "^4.8.3",  
  "typeorm": "^0.3.28"  
},
```

Smart TV (Angular)

Dependencias principales:

Dependencia	Función
@angular/service-worker	Funcionalidad PWA
socket.io-client	Actualización en tiempo real
rxjs	Programación reactiva
Angular Router	Navegación
Angular Forms	Formularios

```
"private": true,  
"dependencies": {  
  "@angular/animations": "^18.2.0",  
  "@angular/common": "^18.2.0",  
  "@angular/compiler": "^18.2.0",  
  "@angular/core": "^18.2.0",  
  "@angular/forms": "^18.2.0",  
  "@angular/platform-browser": "^18.2.0",  
  "@angular/platform-browser-dynamic": "^18.2.0",  
  "@angular/router": "^18.2.0",  
  "@angular/service-worker": "^18.2.0",  
  "rxjs": "~7.8.0",  
  "socket.io-client": "^4.8.3",  
  "tslib": "^2.3.0",  
  "zone.js": "~0.14.10"  
},  
"devDependencies": {  
  "@angular-devkit/build-angular": "^18.2.21",  
  "@angular/cli": "^18.2.21",  
  "@angular/compiler-cli": "^18.2.0",  
  "@types/jasmine": "~5.1.0",  
  "jasmine-core": "~5.2.0",  
  "karma": "~6.4.0",  
  "karma-chrome-launcher": "~3.2.0",  
  "karma-coverage": "~2.2.0",  
  "karma-jasmine": "~5.1.0",  
  "karma-jasmine-html-reporter": "~2.1.0",  
  "typescript": "~5.5.2"  
}
```

Procedimiento de instalación

Para reproducir el entorno de desarrollo es necesario seguir el siguiente procedimiento:

1. Instalar Flutter SDK versión 3.44.0.
2. Instalar Android Studio con los plugins Flutter y Dart.
3. Instalar Visual Studio Code.
4. Clonar el repositorio del proyecto.
5. Configurar las variables de entorno del backend.
6. Ejecutar flutter pub get en la aplicación móvil.
7. Ejecutar flutter pub get en la aplicación Wear OS.
8. Ejecutar npm install en el backend.
9. Ejecutar npm install en la aplicación Smart TV.
10. Crear los emuladores de teléfono y Wear OS desde Android Studio.
11. Configurar Chrome DevTools para la simulación de Smart TV.
12. Ejecutar los tres proyectos simultáneamente para validar el ecosistema.

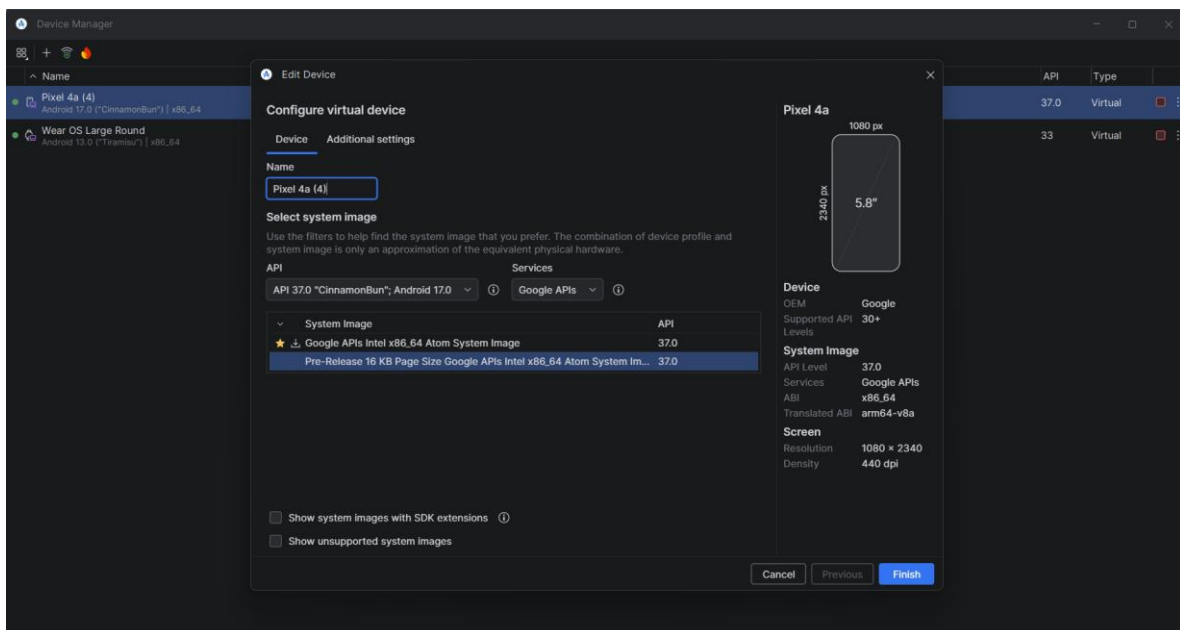
SA.6.B Configuración de emuladores

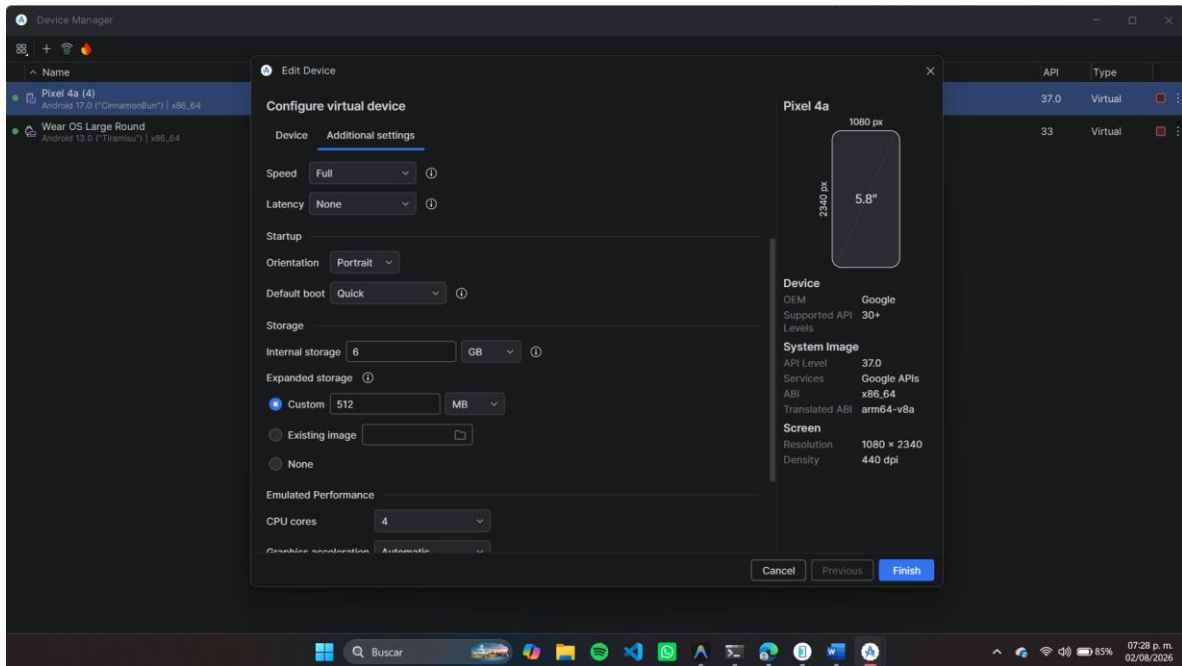
Emulador de teléfono

Para las pruebas de la aplicación móvil se utilizó el siguiente dispositivo virtual:

- Modelo: Google Pixel 4a
- Sistema operativo: Android 17
- API Level: 37
- Arquitectura: x86_64
- Procesador: 4 núcleos
- RAM asignada: 2048 MB
- Resolución: 1080 × 2340 píxeles
- Densidad: 440 dpi
- Almacenamiento interno: 6 GB

Este emulador permitió validar el funcionamiento completo de la aplicación móvil, incluyendo autenticación, consumo de API, sincronización con el reloj y comunicación con la PWA.





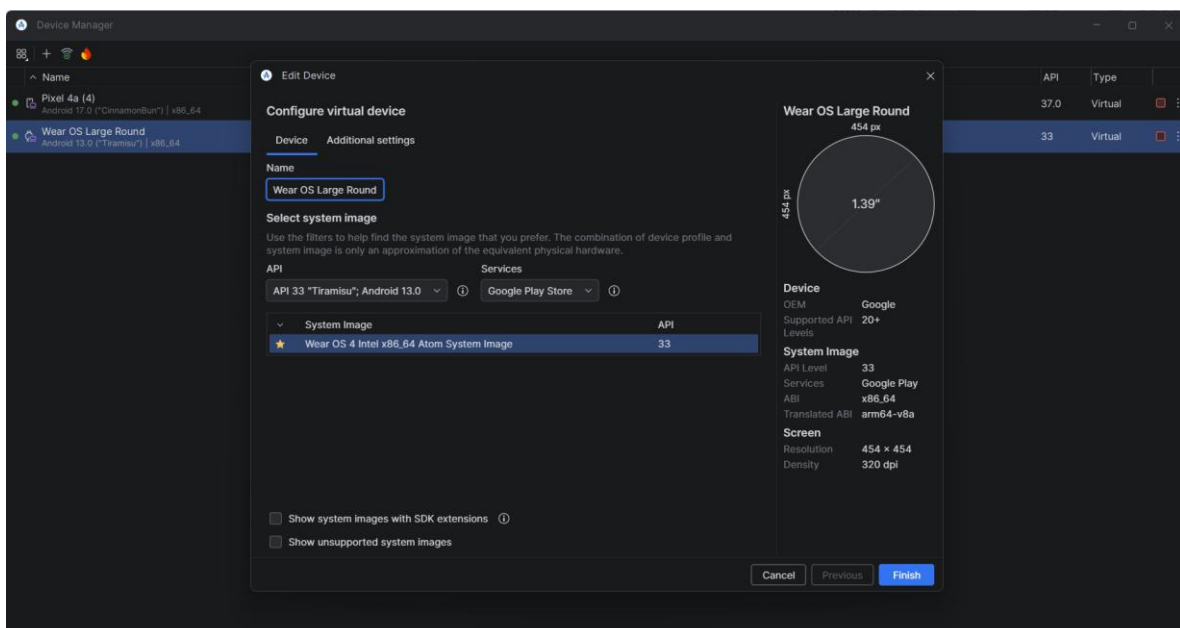
Emulador Wear OS

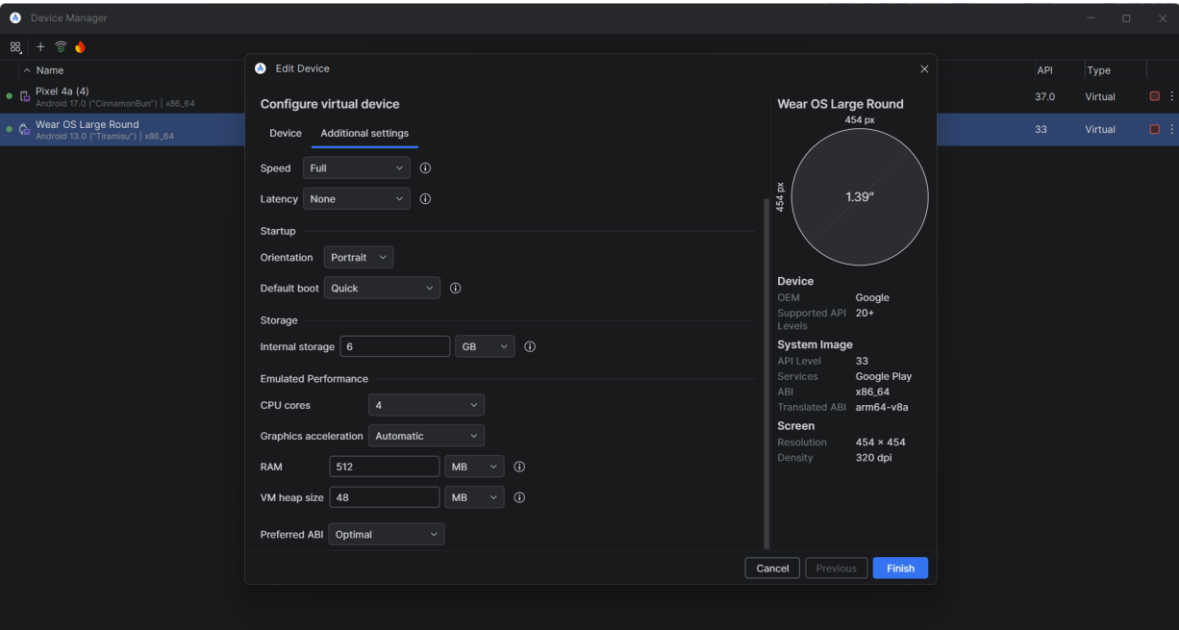
La aplicación para Wear OS fue validada mediante un emulador oficial de Android Studio.

Configuración utilizada:

- Modelo: Wear OS Large Round
- Forma: Circular
- Android Wear OS 4
- API Level: 33
- Arquitectura: x86_64
- Procesador: 4 núcleos
- RAM asignada: 512 MB
- Resolución: 454 × 454 píxeles
- Pantalla de 1.39 pulgadas

Este emulador permitió validar la interfaz del reloj, el simulador de sensores y la comunicación con la aplicación del teléfono.





Emulación de Smart TV

La Smart TV fue simulada utilizando Chrome DevTools.

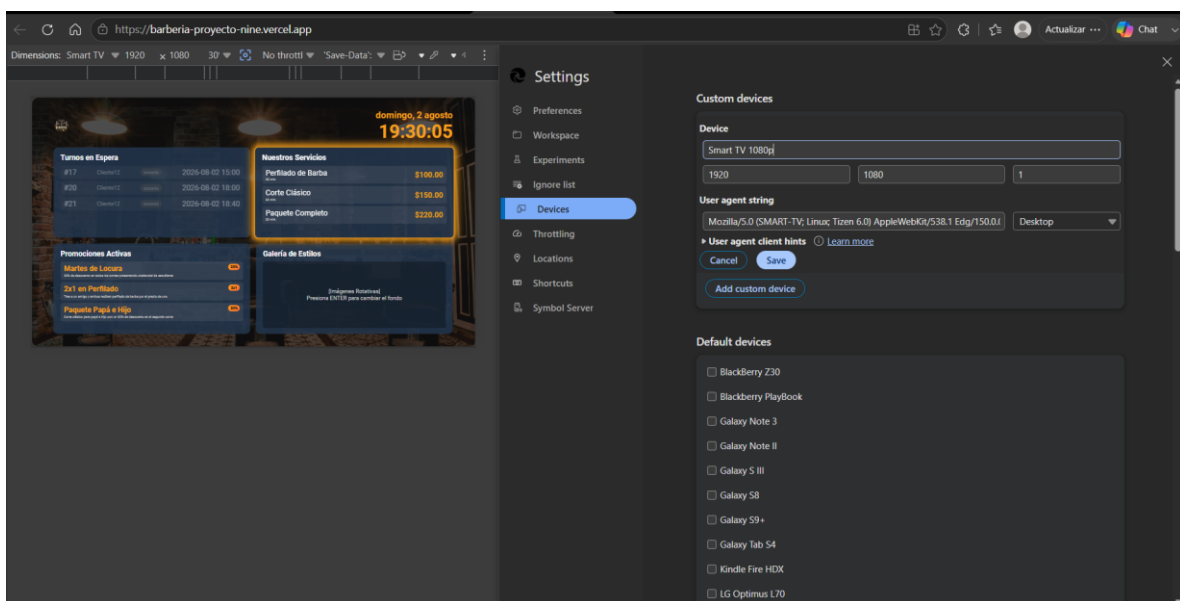
Configuración utilizada:

- Perfil personalizado: Smart TV 1080p
- Resolución: 1920 × 1080
- Relación de aspecto: 16:9
- Escala: 1
- User Agent:

Mozilla/5.0 (SMART-TV; Linux; Tizen 6.0) AppleWebKit/538.1 (KHTML, like Gecko) Version/6.0 Safari/538.1

- Tipo de dispositivo: Desktop

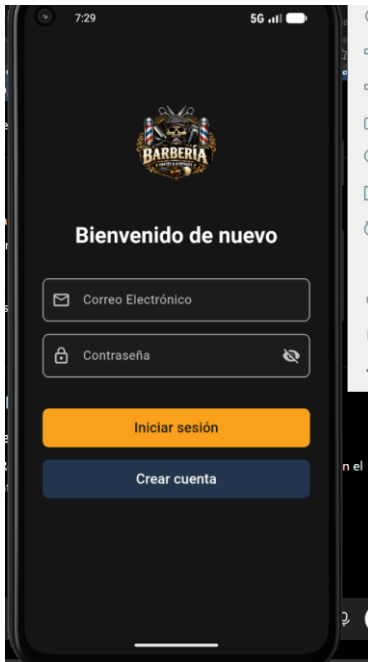
Esta configuración permitió validar la interfaz 10-Foot, la navegación mediante D-pad y el comportamiento de la PWA sobre una resolución Full HD.



Evidencias de funcionamiento

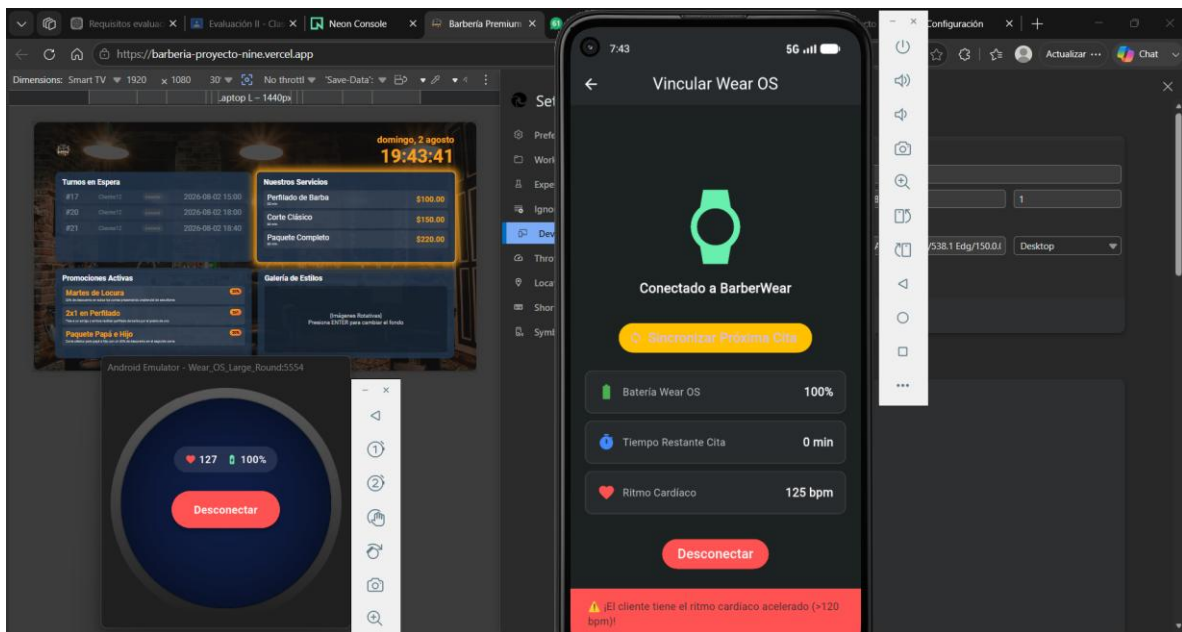
Como evidencia del correcto funcionamiento del ecosistema se incluyen capturas de pantalla de:

- Aplicación del teléfono en ejecución.
- Aplicación Wear OS en ejecución.
- PWA Smart TV en funcionamiento.
- Los tres dispositivos ejecutándose simultáneamente.





Los tres funcionando al mismo tiempo



Problemas encontrados (Troubleshooting)

Problema 1. Variables de entorno en Render

Durante el despliegue del backend en Render, la aplicación no lograba establecer comunicación con el servidor debido a que no se habían configurado correctamente las variables de entorno.

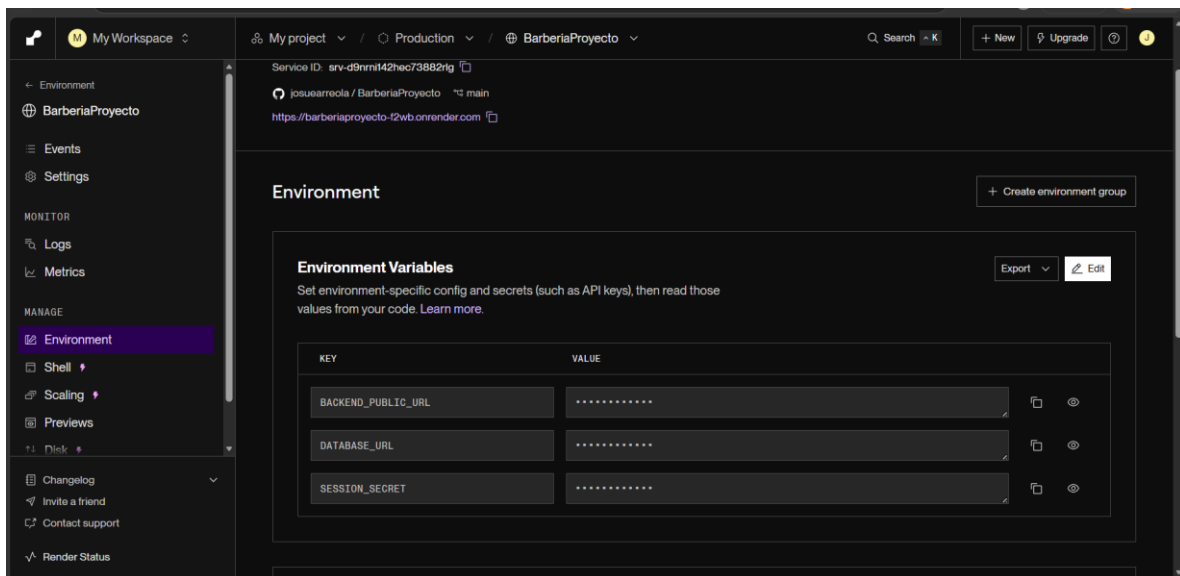
Las variables faltantes fueron:

- BACKEND_PUBLIC_URL
- SESSION_SECRET

Como consecuencia, la aplicación móvil no podía consumir la API y el despliegue del backend era rechazado.

Solución implementada

Se agregaron ambas variables de entorno dentro del panel de configuración de Render y posteriormente se realizó un nuevo despliegue del servicio, logrando establecer correctamente la comunicación entre el backend y las aplicaciones cliente.



Problema 2. Emulación de Bluetooth Low Energy

Los emuladores oficiales de Android Studio no incorporan una antena Bluetooth física, por lo que no es posible realizar pruebas reales de comunicación BLE entre el teléfono y Wear OS.

Solución implementada

Se desarrolló una arquitectura híbrida:

- La implementación BLE completa permanece integrada mediante flutter_blue_plus, flutter_ble_peripheral y ble_peripheral, lista para ejecutarse en dispositivos físicos.
- Para las pruebas realizadas en emuladores se implementó un canal de comunicación equivalente mediante Socket.IO, permitiendo validar el flujo de sincronización en tiempo real entre Wear OS y el teléfono sin requerir hardware físico.

De esta manera fue posible comprobar el funcionamiento del ecosistema completo durante las pruebas de integración.

Conclusiones

La correcta configuración de las herramientas de desarrollo y de los emuladores permitió implementar y validar el ecosistema completo compuesto por la aplicación móvil, la aplicación Wear OS y la PWA para Smart TV. A pesar de las limitaciones propias de los emuladores, especialmente en la comunicación Bluetooth Low Energy, fue posible realizar pruebas funcionales mediante una arquitectura híbrida basada en Socket.IO, manteniendo la implementación BLE preparada para dispositivos reales.

El entorno de desarrollo documentado es completamente reproducible, permitiendo que otro desarrollador pueda instalar las herramientas necesarias, configurar los proyectos y ejecutar el ecosistema siguiendo los pasos descritos en este documento, cumpliendo con los requisitos establecidos en la rúbrica de evaluación SA.6.

Justificación del uso de Inteligencia Artificial

Durante la elaboración del software y la documentación del proyecto se utilizó una herramienta de Inteligencia Artificial (ChatGPT, OpenAI) como apoyo en la detección y solución de problemas relacionados con el código, análisis de errores, consulta de posibles soluciones técnicas y mejora de la redacción del contenido.

La Inteligencia Artificial fue utilizada como una herramienta complementaria durante el proceso de desarrollo, pero no realizó la programación del sistema ni sustituyó el análisis, la lógica de implementación, las pruebas o la validación del funcionamiento del software. La arquitectura, desarrollo, decisiones técnicas y comprobaciones del sistema fueron realizadas por el autor del proyecto.

En la elaboración del documento, la información y contenido técnico fueron redactados previamente por el autor con base en el trabajo realizado, evidencias obtenidas y pruebas ejecutadas. Posteriormente, la Inteligencia Artificial fue utilizada únicamente para apoyar en la corrección de redacción, ortografía, estructura y claridad del texto, manteniendo la responsabilidad del contenido final en el autor.